Санкт-Петербургский государственный университет

Программная инженерия

Группа 24.М71-мм

# Исследование и применение больших языковых моделей для автоматизации построения исполняемых сред программных проектов

***Ван Цзыханъ***

Отчёт по учебной практике
в форме «Эксперимент»

Научный руководитель:
ст. преподаватель кафедры ИАС, к.ф.-м.н. Азимов Р. Ш.

Санкт-Петербург
2026

# Оглавление

**Введение** **3**

**1. Постановка задачи** **5**

**2. Обзор** **6**

2.1. Обзор существующих решений . . . . . 6

2.2. Обзор используемых технологий . . . . . 7

**3. Описание решения** **11**

3.1. Архитектура системы . . . . . 11

3.2. Изолированная песочница и механизм управления сессиями 12

3.3. Реализация интеллектуального движка принятия решений 14

3.4. Реализация инструментария верификации среды . . . . . 17

**4. Экспериментальное исследование** **19**

4.1. Условия эксперимента и описание данных . . . . . 19

4.2. Описание набора данных . . . . . 19

4.3. Исследовательские вопросы . . . . . 20

4.4. Метрики оценки . . . . . 20

4.5. Процедура проведения эксперимента . . . . . 21

4.6. Сводные результаты эксперимента . . . . . 21

4.7. Анализ результатов . . . . . 22

**Заключение** **27**

**Список литературы** **29**

# Введение

В последние годы крупные языковые модели (LLM) продемонстрировали значительный прогресс в задачах генерации кода и написания патчей [1]. Такие системы, как SWE-agent [2], OpenHands [3] и Copilot [4], способствуют парадигмальному сдвигу в области программной инженерии (Software Engineering, SE); их глубокая интеграция в процессы разработки (обозначаемая как LLM4SE) становится новой устойчивой тенденцией. Тем не менее переход LLM4SE от «статической генерации кода» к «динамическому решению задач» сталкивается с принципиальным узким местом — отсутствием высококачественных, интерактивных сред для обучения и оценки. Несмотря на относительную зрелость LLM в задачах статической генерации кода, их способность к автоматизированной динамической верификации остаётся существенно ограниченной. Как отмечают Gao и соавт. [5], реальные программные проекты характеризуются сложной проектно-ориентированной структурой знаний и зависимостей; даже опытные разработчики вынуждены затрачивать значительные ресурсы на конфигурирование и отладку среды. Именно эта жёсткая зависимость от воспроизводимой исполняемой среды является ключевым фактором, препятствующим замыканию цикла (end-to-end) выполнения реальных задач программной инженерии средствами LLM.

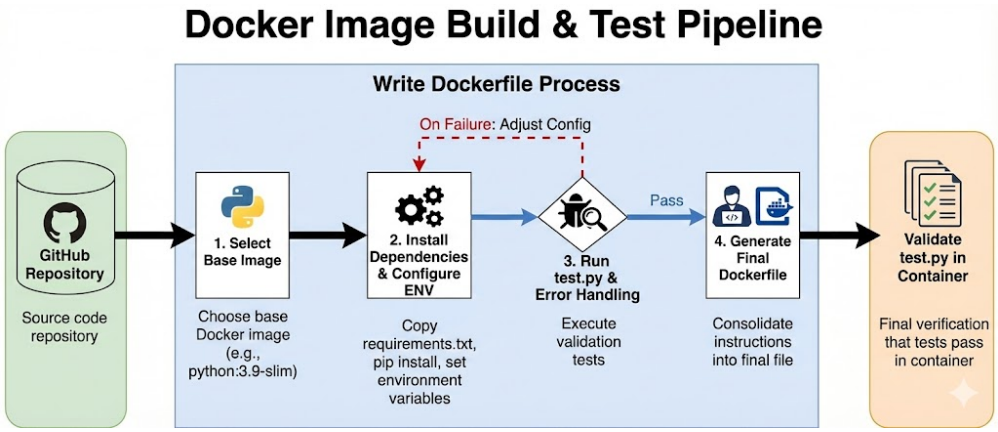


Рис. 1: Docker Image Build and Test Pipeline

Типичный процесс построения среды представлен на рис. 1. В настоящее время автоматизация данного процесса сталкивается с двумя

фундаментальными разрывами. Первый — разрыв между статическими эталонными бенчмарками и динамическими требованиями реальных проектов. Хотя существуют высококачественные наборы для оценки, такие как SWE-bench [1] и SWE-Gym [6], их базовая инфраструктура в значительной степени зависит от ручной предварительной сборки и сопровождения, что ограничивает масштабируемость и затрудняет адаптацию к стремительному росту экосистемы открытого программного обеспечения. Второй — разрыв между детерминированностью и адаптивностью. Современные автоматизированные инструменты (например, repo2docker [7]) преимущественно опираются на предопределённые эвристические правила; при возникновении нетипичных ошибок — конфликтов зависимостей, несовместимости версий и т.п. — они, ввиду отсутствия механизмов семантической интерпретации ошибок и автономного восстановления, часто завершаются неудачей. В то же время новые программные агенты (например, SWE-agent [2]), несмотря на наличие интерфейса командной строки, не обладают специально спроектированным пространством действий для разрешения системных зависимостей и конфликтов, что в сложных конфигурационных сценариях приводит к неэффективному перебору и даже к «загрязнению» состояния среды.

Для преодоления указанных ограничений в настоящей работе предлагается и реализуется фреймворк автоматизированного построения программной среды на основе самоэволюционирующего интеллектуального агента. Ключевая инновация подхода заключается в трансформации традиционного статического и детерминированного процесса конфигурирования в динамический замкнутый цикл «восприятие — рассуждение — исполнение — эволюция». За счёт глубокой интеграции возможностей крупной языковой модели по пониманию кода и логическому выводу разработанный агент способен активно воспринимать мультимодальную обратную связь в ходе сборки (журналы, коды ошибок, изменения состояния файлов), динамически диагностировать первопричины сбоев и самостоятельно планировать и выполнять стратегии восстановления. Это позволяет синтезировать стабильные и воспроизводимые конфигурации среды без необходимости ручного вмешательства.

# 1. Постановка задачи

Целью данной работы является разработка прототипа системы на базе автономных агентов с использованием больших языковых моделей (LLM), предназначенной для сквозного (end-to-end) автоматизированного развертывания исполняемых сред Docker из репозиториев исходного кода. Система имитирует итеративный процесс «проб и ошибок», характерный для опытных разработчиков, для решения проблем сложности зависимостей, неопределенности путей сборки и высоких затрат на ручную конфигурацию при динамической оценке крупномасштабных кодовых баз.

1. Разработка двухуровневой архитектуры системы: создание изолированной «песочницы» Docker для безопасного выполнения команд и внешнего управляющего модуля на базе LLM, который анализирует ошибки из консоли, принимает решения о дальнейших действиях и фиксирует состояние процесса.

2. Формирование набора типовых операций для настройки среды: систематизация основных команд (установка зависимостей, конфигурация системных путей, отладка компиляции), позволяющих агенту автономно подбирать нужные инструкции и находить рабочие пути сборки для различных репозиториев.

3. Реализация механизма автоматического отката и синтеза Dockerfile: внедрение функции возврата к «чистому» состоянию системы при возникновении критических ошибок для предотвращения её загрязнения, а также автоматическое преобразование успешных цепочек действий в готовый к использованию Dockerfile-скрипт.

4. Проведение экспериментов на реальных проектах: тестирование системы на базе репрезентативной выборки Python-репозиториев из GitHub и выполнение бенчмаркинга для оценки её эффективности.

# 2. Обзор

Данный раздел ставит целью проведение систематизированного обзора современных исследований и технологических предпосылок в области автоматизированной сборки программных сред и верификационного тестирования. В настоящее время научное сообщество и инженерно-программная сфера сталкиваются с серьёзной «проблемой воспроизводимости»: исследователи зачастую не могут запустить исходный код авторов или испытывают значительные трудности при попытках трансформировать описанные в публикациях методики в эквивалентные исполняемые среды.

## 2.1. Обзор существующих решений

Прежде всего, наиболее базовым подходом является ручное следование проектной документации (Manual Following). Разработчик, изучая неструктурированные пояснительные файлы, такие как README, применяет парадигму проб и ошибок (trial-and-error) для самостоятельного разрешения конфликтов зависимостей. Однако данный способ не только чрезвычайно трудоёмок, но и полностью лишён масштабируемости при необходимости обработки большого числа репозиториев в рамках задач массового анализа и извлечения данных.

Во-вторых, ряд работ предпринимает попытку повторного использования существующих конвейеров непрерывной интеграции и доставки (CI/CD). Хотя подобные пайплайны (например, GitHub Actions) обеспечивают определённый уровень автоматизации, далеко не все научные проекты оснащены подобной инфраструктурой. Кроме того, их функционирование существенно зависит от замкнутых экосистем конкретных платформ, а гетерогенность конфигурационных сценариев и технологических стеков между различными платформами приводит к высоким издержкам миграции и повышенной сложности эксплуатации.

Наконец, в последние годы широкое распространение получили автоматизированные методы, основанные на эвристических сценариях (heuristic-based automation), типичным представителем которых являет-

ся repo2docker [7]. Данный инструмент стремится детерминированно воссоздавать Docker-образы путём автоматического анализа стандартных конфигурационных файлов в репозитории (например, requirements.txt или environment.yml).

Тем не менее, подобные эвристические схемы построения среды демонстрируют выраженную хрупкость при работе с крупными и сложными репозиториями. Поскольку их базовая логика опирается на заранее заданные статические правила сопоставления, при возникновении критических ошибок — таких как отсутствие зависимостей, несовместимость версий или конфликты системных библиотек — система, как правило, не способна интерпретировать глубинную семантику сообщений об ошибках. По своей сути такие инструменты представляют собой детерминированные неконтурные (non-closed-loop) системы, лишённые механизмов автономного рассуждения на основе обратной связи выполнения. В результате при сбоях сборки они ограничиваются выводом объёмных журналов ошибок и прерыванием процесса.

В подобных ситуациях система не обладает способностью к динамической корректировке стратегии и восстановлению конфигурации с учётом конкретных исключений времени выполнения, что обуславливает необходимость значительного ручного вмешательства для диагностики и устранения неисправностей. Чрезмерная зависимость от стандартных конфигурационных файлов в сочетании с отсутствием механизмов самовосстановления существенно ограничивает успешность и устойчивость традиционных автоматизированных инструментов при работе с нестандартными или эволюционирующими кодовыми базами.

## 2.2. Обзор используемых технологий

С целью построения интеллектуальной системы, способной к автономному восстановлению и верификации среды исполнения, в настоящем проекте интегрированы технологии контейнеризации, интеллектуальный агент на основе крупных языковых моделей (LLM), а также принципы автоматизированного тестирования. В совокупности они формируют

замкнутый конвейер с возможностями «восприятия — рассуждения — исполнения».

### 2.2.1. Технологии контейнеризации и Docker

Контейнеризация является фундаментом построения воспроизводимых исследовательских сред. Docker [8], используя механизм облегчённой виртуализации, позволяет инкапсулировать программное обеспечение вместе с полным стеком зависимостей в изолированные «образы», обеспечивая тем самым консистентность выполнения на различных аппаратных платформах и операционных системах.

В научном контексте ключевая ценность Docker заключается в трансформации статического репозитория исходного кода в исполняемую динамическую программную среду, что существенно снижает объём повторяющихся усилий по конфигурированию среды со стороны исследователей. Вместе с тем эмпирические исследования показывают, что большинство учёных не обладают достаточной компетенцией для самостоятельной разработки Dockerfile, а традиционные инструменты, такие как repo2docker, в значительной степени опираются на наличие стандартизированных конфигурационных файлов (например, requirements.txt или setup.py) для детерминированного воссоздания среды.

В рамках данного проекта Docker используется в качестве низкоуровневого исполнительного механизма, тогда как автоматизированный интеллектуальный конвейер призван устранить разрыв между потребностями научных проектов и сложностью системной конфигурации.

### 2.2.2. LLM-ориентированный AI Agent как механизм рассуждения

В качестве центрального компонента принятия решений в проекте используется AI Agent, функционирующий на основе крупной языковой модели (LLM) и дополненный разработанным авторами инструментарием, обеспечивающим глубокое взаимодействие с кодовой базой.

В отличие от традиционных жёстко заданных сценариев, логика

агента не ограничивается заранее запрограммированными правилами. Он опирается на семантические возможности LLM для интерпретации описаний на естественном языке и сложных журналов ошибок сборки (stderr), что позволяет преодолевать ограничения автоматизированных систем, неспособных детерминированно обрабатывать неконсистентные или неполные конфигурационные описания.

Интеграция собственных низкоуровневых инструментов предоставляет агенту прямой доступ к чтению и модификации исходного кода проекта. Это позволяет ему, исходя из контекста ошибки (error context), осуществлять целенаправленное логическое рассуждение, выявлять глубинные конфликты зависимостей и выполнять их корректировку.

### 2.2.3. Механизм итеративной верификации, управляемый обратной связью

Для предотвращения феномена галлюцинаций (hallucination) со стороны AI Agent в процессе конфигурирования среды модельные предсказания не рассматриваются в качестве критерия успешности сборки. Вместо этого внедрён разработанный авторами эмпирический скрипт проверки — runtest.py.

Ключевой принцип данного механизма заключается в использовании фактических результатов выполнения скрипта внутри контейнера (возвратный код, сообщения об ошибках времени выполнения и т.д.) в качестве единственного источника объективной истины (ground truth). Система принудительно ориентирует агента на итеративную корректировку конфигурации на основе объективной обратной связи выполнения, а не на основе субъективных предположений модели.

В случае неуспешного выполнения runtest.py полученная диагностическая информация инициирует повторный цикл анализа кода и конфигурации с последующим целевым внесением изменений. Подобная верификационная парадигма, основанная на реальном исполнении, обеспечивает не только синтаксическую корректность среды, но и её функциональную пригодность для проведения научных экспериментов. Тем самым существенно повышается успешность и устойчивость авто-

матизированного построения среды для сложных гетерогенных кодовых баз.

# 3. Описание решения

В данной главе рассматривается реализация интеллектуального агента автоматизированной настройки среды Python на основе большой языковой модели (LLM) (далее — «Агент»).

Система предназначена для решения задач сборки и разрешения зависимостей в крупных репозиториях кода, функционирующих в гетерогенных средах. В её основе лежит замкнутый цикл «наблюдение — принятие решения — выполнение — проверка», обеспечивающий последовательную диагностику и корректировку конфигурации.

Архитектура системы имеет модульную структуру и включает четыре основных компонента: изолированную песочницу выполнения, механизм интеллектуального принятия решений, инструменты проверки среды и систему управления состоянием.

## 3.1. Архитектура системы

В данной системе используется многоуровневая архитектура, обеспечивающая разделение функций и безопасную изоляцию. Архитектура состоит из трёх уровней:

1. **Уровень инфраструктуры (Infrastructure Layer).** Основан на технологии Docker-контейнеров для создания полностью изолированной песочницы выполнения. Данный уровень обеспечивает физическую синхронизацию целевого репозитория с контейнерной средой через механизм монтирования томов (Volume Mounting), что гарантирует отсутствие вмешательства процесса конфигурации в основную систему. Кроме того, данный уровень поддерживает создание снимков состояния контейнерной среды и откат к ним, обеспечивая безопасность при исследовательских операциях.

2. **Уровень сервисов инструментов (Tool Service Layer).**

   Инкапсулирует ряд стандартизированных атомарных операций, ориентированных на агентов. Конкретные инструменты включа-

ют: инструмент проверки окружения (`runtest.py`), инструмент обнаружения конфликтов зависимостей (`check_deps.py`), а также инструмент редактирования кода (`code_edit.py`). Данный уровень абстрагирует сложные системные вызовы и команды скриптов в интерфейсы инструментов с чёткой семантикой и стабильными входными и выходными данными.

3. **Уровень интеллектуального принятия решений (Intelligent Decision Layer).**

   Является ядром системы управления и управляется большой языковой моделью. На данном уровне посредством тщательно спроектированного инжиниринга промтов (Prompt Engineering) реализована логика принятия решений на основе конечного автомата, позволяющая агенту автономно планировать и выполнять полный процесс конфигурации.

## 3.2. Изолированная песочница и механизм управления сессиями

В процессе автоматизированного построения окружения, для обеспечения баланса между гибкостью действий интеллектуального агента и безопасностью физической среды, была разработана изолированная песочница и механизм управления сессиями на базе Docker.

### 3.2.1. Инкапсуляция состояния сессии Docker

Система реализует стандартный интерфейс взаимодействия интеллектуального агента с физической средой через класс `DockerSession`. Ключевой особенностью данного класса является внедрение **механизма динамической инъекции переменных окружения**: перед выполнением каждой инструкции система автоматически добавляет префиксы конфигурации переменных `PATH` и `PYTHONPATH`.

Такая архитектура гарантирует, что инструментарий, установленный агентом через `uv` или `pip`, а также локальные модули внутри репозито-

рия будут немедленно доступны в неинтерактивных сессиях оболочки (shell). Данный подход позволяет эффективно устранить «ложные сбои конфигурации», вызванные ошибками поиска путей, и значительно повышает согласованность и надежность восприятия среды агентом.

### 3.2.2. Механизм защитных снимков и откатов

Для предотвращения необратимых повреждений среды (таких как каскадный сбой зависимостей или случайное удаление системных файлов), вызванных действиями агента, в системе реализован механизм дефенсивного управления на основе команды `docker commit`, включающий в себя три основных аспекта:

1. **Семантическое распознавание команд**: Система использует фильтр команд для предварительного аудита операций, разделяя их на «наблюдения только для чтения» (например, `ls`, `cat` и другие операции из «белого списка» безопасности) и «действия, изменяющие состояние» (такие как `pip install`, `rm -rf`, `sed -i`).

2. **Атомарная защита состояния**: Перед выполнением любой команды, изменяющей состояние, система автоматически создает снимок (snapshot). Если после выполнения команды возвращается ненулевой код завершения, система принудительно отбрасывает поврежденный слой и выполняет откат к последнему стабильному образу, обеспечивая обратимость путей конфигурации и атомарность среды.

3. **Ограниченная стратегия отката**: Механизм отката позиционируется как «последнее средство» при возникновении критических ошибок. Чтобы предотвратить зацикливание агента и злоупотребление данной функцией при неэффективных попытках, количество откатов в рамках одной сессии строго ограничено двумя разами. Данное ограничение вынуждает агента корректировать стратегию принятия решений на более высоком уровне (например, пробовать

сброс конфигурации или смену базовой версии Python) в случае неудачи.

## 3.3. Реализация интеллектуального движка принятия решений

Интеллектуальный движок принятия решений является «центром управления» системы, отвечающим за интерпретацию сложных ошибок конфигурации и генерацию соответствующих инструкций по исправлению. Данный раздел описывает формальную математическую модель, поэтапный процесс рассуждения, управление контекстом и механизмы вмешательства при обнаружении зацикливаний.

### 3.3.1. Формальное определение процесса принятия решений

Для обеспечения точности взаимодействия Агента с песочницей процесс конфигурации моделируется как итеративный процесс перехода состояний. Глобальное состояние системы $S$ на шаге $t+1$ определяется следующим образом:

$$S_{t+1} = \mathcal{T}(S_t, a_t) = (\mathcal{T}_E(E_t, a_t),\ H_t \oplus (a_t, O_t)) \tag{1}$$

Где приняты следующие обозначения:

- $S_t$ — глобальное состояние в момент $t$, включающее физическую среду $E_t$ и траекторию истории диалога $H_t$.
- $a_t = \pi_\theta(H_t)$ — действие, генерируемое стратегией $\pi_\theta$ (моделью LLM с параметрами $\theta$), представляющее собой bash-команду или модификацию файла (diff).
- $O_t = \mathcal{O}(E_t, a_t)$ — наблюдение, полученное из песочницы, состоящее из стандартного вывода, потока ошибок и кода завершения (Exit Code).

- $\mathcal{T}_E$ — функция перехода физического состояния, отражающая изменения файловой системы или дерева зависимостей внутри Docker-контейнера.

- $\oplus$ — оператор конкатенации последовательностей для обновления контекста новыми записями $(a_t, O_t)$.

### 3.3.2. Поэтапное рассуждение на основе конечного автомата

Для минимизации случайности вывода и исключения галлюцинаций LLM, система через системный промпт (System Prompt) навязывает Агенту логику четырехэтапного конечного автомата:

1. **Этап аудита среды (Audit)**: Агент выполняет исследовательские команды (например, `ls -la`), идентифицирует метаданные (такие как `requirements.txt`, `pyproject.toml`) и определяет приоритетность систем сборки.

2. **Этап предварительной проверки (Pre-check)**: На основе аудита Агент оценивает потенциальные конфликты версий. Система придерживается принципа приоритета существующих конфигурационных файлов над созданием новых.

3. **Этап атомарной установки (Execution)**: Выполнение команд установки. Система отдает приоритет высокопроизводительным менеджерам пакетов (например, `uv`). При возникновении конфликтов Агент принимает решение об ослаблении ограничений версий (Relaxation).

4. **Этап финальной верификации (Verification)**: Вызов инструмента `runtest.py`. Агент анализирует специфические коды завершения (2, 4 или 5) и на основе этих сигналов принимает решение о завершении задачи или переходе к новой итерации.

### 3.3.3. Многоходовый диалог и управление контекстом

Агент поддерживает последовательность истории сообщений $H_t$ для обеспечения логической связности. Для решения проблем переполнения токенов и «размывания» контекста реализован механизм интеллектуального подавления шума в выводе команд.

При выполнении команд установки (`pip install` или `uv sync`), которые генерируют тысячи строк логов, движок принятия решений выполняет обрезку наблюдения $O_t$ в режиме реального времени. Система извлекает только критические данные: стеки ошибок (Traceback), сводки конфликтов версий и итоговые отчеты об успехе. Это гарантирует, что модель всегда сосредоточена на ключевых изменениях физической среды.

### 3.3.4. Обнаружение зацикливаний и усиление обратной связи

В процессе конфигурации Агент может попасть в бесконечный цикл «ошибка-повтор-ошибка» из-за неразрешимых конфликтов зависимостей. Система отслеживает последовательность действий $a_{t-k:t}$ и признаков наблюдений $O_{t-k:t}$ для идентификации зацикливания:

$$\text{is_loop} = \mathbb{I}\left(\text{Count}(\text{Signature}(O_{t-k:t})) \geq 3\right) \tag{2}$$

При обнаружении трехкратного повторения идентичной ошибки система перехватывает поток управления и внедряет в `System Message` высокоприоритетный **сигнал интервенции**. Данная инструкция явно указывает на состояние цикла и предписывает Агенту сменить стратегию (например, перейти от исправления зависимостей к настройке путей `PYTHONPATH` или пропуску поврежденных тестовых файлов), что обеспечивает выход из логического тупика.

## 3.4. Реализация инструментария верификации среды

Инструментарий верификации является критическим диагностическим компонентом системы, который преобразует неструктурированные сообщения об ошибках терминала в высокоуровневые логические сигналы для Агента. Основным инструментом этого слоя является `runtest.py`, который выполняет роль интеллектуальной надстройки над `pytest`.

### 3.4.1. Интеллектуальный сборщик тестов (runtest.py)

В отличие от стандартного запуска тестов, `runtest.py` использует режим `pytest --collect-only` для превентивной проверки целостности графа зависимостей. В инструменте реализована трехступенчатая система классификации ошибок:

1. **Проверка существования в PyPI**: Перед тем как рекомендовать Агенту установку недостающего модуля, инструмент выполняет асинхронный запрос к API PyPI (`https://pypi.org/pypi/{package}/json`). Это позволяет отсечь ошибки, вызванные опечатками в коде или использованием внутренних приватных библиотек, которые невозможно загрузить из публичных репозиториев.

2. **Идентификация локальных модулей**: Инструмент анализирует структуру каталогов `/repo`. Если импортируемый модуль (например, `project.utils`) совпадает с именем подкаталога или файла в корне репозитория, он классифицируется как локальный. В этом случае Агенту предлагается настроить переменную `PYTHONPATH` вместо попытки внешней установки, что предотвращает конфликт между локальным кодом и одноименными пакетами в облаке.

3. **Диагностика совместимости версий**: Система анализирует стек вызовов (Traceback) для различения «ошибки импорта из-за

отсутствия пакета» и «ошибки атрибута внутри установленного пакета». Последнее часто сигнализирует о несоответствии версий (Version Mismatch), когда API библиотеки изменилось, что требует обновления или отката конкретной зависимости.

### 3.4.2. Система дифференцированных кодов завершения

Для обеспечения эффективного взаимодействия в парадигме «машина-машина» (M2M), инструмент верификации передает результаты диагностики через систему специфических кодов завершения (Exit Codes). Это позволяет Агенту мгновенно определять категорию проблемы без необходимости полного семантического анализа логов.

В разработанной системе каждый код соответствует определенному сценарию: код 0 подтверждает успешную конфигурацию; код 2 указывает на обнаружение локального модуля, требующего настройки `PYTHONPATH` или редактируемой установки; код 3 сигнализирует о логических ошибках в коде или глубоких конфликтах версий; код 4 идентифицирует отсутствие пакета в репозитории PyPI; а код 5 сообщает об отсутствии доступных тестов в проекте. Данный механизм гарантирует, что Агент получает не просто отчет о сбое, а структурированный сигнал для выбора следующей стратегии действия, что существенно сокращает количество итераций, необходимых для успешной сборки среды.

# 4. Экспериментальное исследование

## 4.1. Условия эксперимента и описание данных

Для обеспечения воспроизводимости результатов и объективности оценки разработанного подхода экспериментальное исследование проводилось в строго контролируемых условиях на базе специально сформированного набора тестовых данных.

**Программно-аппаратная среда.** Основные характеристики окружения, в котором проводилась апробация системы, представлены ниже:

- Аппаратное обеспечение: процессор Intel Core i7-12700H (14 ядер) и 32 ГБ оперативной памяти стандарта DDR5.
- Операционные системы хоста: Windows 11 и Ubuntu 22.04.
- Изоляция окружения: Docker-контейнер на базе образа `python:3.10` (4 vCPU, 8 ГБ RAM), что соответствует параметрам типовых узлов CI/CD.
- Технологический стек: в качестве интеллектуального ядра использовалась модель DeepSeek-V3 через программный интерфейс (API) с фиксированным значением температуры ($T = 0$).

## 4.2. Описание набора данных

Для проведения эксперимента был сформирован набор из 98 публичных Python-репозиториев с GitHub. Репозитории были отобраны на основе отсутствия готовых инструкций по запуску, наличия декларативных файлов зависимостей и возможности автоматизированной верификации сборки.

Размер отобранных проектов варьировался от 1 000 до 35 000 строк кода. В выборку вошли проекты из различных областей программной инженерии, включая системы обработки данных, библиотеки машинного обучения и консольные утилиты.

**Classification Summary**

| Category | Description | Count |
|---|---|---|
| AI / LLM / Agent frameworks | LLM agents, AI frameworks, model tools | 28 |
| RAG / Vector / GraphRAG | RAG systems, retrieval, embedding | 15 |
| Web / API / FastAPI apps | Web services, API frameworks, backend templates | 11 |
| Dev tools / CLI / DevOps | Dev tools, CLI, automation | 12 |
| Data / ETL / Data tools | Data pipelines, data processing | 9 |
| Security / Test / Eval | Security, testing, evaluation frameworks | 7 |
| Scraping / Automation / Browser | Web scraping, automation | 6 |
| ML / Deep learning | ML tools, training frameworks | 5 |
| Database / DB tools | SQL / Graph DB tools | 3 |
| Other / Misc | Uncategorized | 2 |
| **Total** | | **98** |

Рис. 2: Классификация выбранных репозиториев по типу проектов

## 4.3. Исследовательские вопросы

В рамках экспериментального исследования были сформулированы четыре ключевых вопроса (Research Questions):

- RQ1 (Эффективность): Способна ли предложенная система значимо повысить вероятность успешной конфигурации окружения по сравнению с существующими методами?
- RQ2 (Затраты): Каковы операционные затраты разработанного решения, выраженные во временных и финансовых показателях?
- RQ3 (Границы применимости): Какие характеристики программных проектов являются предикторами отказа системы в автоматической конфигурации?
- RQ4 (Вклад компонентов): Какое влияние оказывают специализированные инструменты мониторинга и правки кода на итоговую результативность системы?

## 4.4. Метрики оценки

Для количественного анализа эффективности используются показатели, представленные в таблице 1.

| Метрика | Описание | Связанный RQ |
|---|---|---|
| Success Rate (SR) | Процент успешно сконфигурированных сред | RQ1, RQ3 |
| Rounds | Среднее число итераций взаимодействия (лимит: 30) | RQ2 |
| Time-to-Success (TTS) | Время от начала работы до успешной верификации | RQ2 |
| Command Distribution | Статистика частоты вызовов различных типов атомарных операций | RQ4 |

Таблица 1: Сводная таблица экспериментальных показателей

## 4.5. Процедура проведения эксперимента

Процесс автоматизированной конфигурации в разработанной системе разделен на четыре последовательных этапа:

Этап 1: Предварительная обработка. Выполняется клонирование репозитория и статический анализ структуры проекта для предварительной оценки объема зависимостей.

Этап 2: Подготовка изолированной среды. Разворачивается Docker-контейнер, выполняется инъекция вспомогательных инструментов (`check_deps.py`, `code_edit.py`, `runtest.py`) и инициализация менеджеров пакетов.

Этап 3: Итерационный цикл конфигурации. Основной этап работы интеллектуального компонента, включающий аудит, выбор стратегии установки, атомарное выполнение команд и верификацию промежуточных состояний.

Этап 4: Финальная верификация. Система считается успешно справившейся с задачей, если тесты завершаются с кодом выхода 0 или 5 и подтверждаются инструментальными проверками.

## 4.6. Сводные результаты эксперимента

Общий коэффициент успеха предложенной системы составил 92.8%. Сравнительный анализ с существующими решениями представлен в таблице 2.

Среднее время успешной настройки одной сессии составило 670.04 секунд. Детальное распределение временных затрат по этапам приведено в таблице 3. Наибольшую долю времени занимает финальная проверка 34.6%) и LLM-инференс (29.7%), тогда как подготовка окружения занимает всего 5.5% общего времени. Эти результаты показывают, что

предложенная система эффективно распределяет ресурсы и большую часть времени использует на ключевые этапы автоматической настройки.

| Method / System | Success Rate (SR) |
|---|---|
| **Proposed System** | **92.8%** |
| Pipreqs-based method (Baseline) | 16.67% |
| LLM generator[9] | 22.1% |
| SWE-agent[9] | 9.0% |

Таблица 2: Сравнение эффективности различных подходов к конфигурации

Среднее время успешной настройки составило 486.5 секунд. Детальное распределение временных затрат приведено в таблице 3.

| Stage | Avg. Time (s) | Share (%) |
|---|---|---|
| Env setup | 37.04 | 5.5% |
| LLM inference | 199.17 | 29.7% |
| Dep install | 149.40 | 22.3% |
| Other commands | 52.35 | 7.8% |
| Final verification | 232.08 | 34.6% |
| Total (TTS) | 670.04 | 100% |

Таблица 3: Распределение временных затрат на одну сессию

## 4.7. Анализ результатов

### 4.7.1. RQ1: Эффективность конфигурации

Результаты эксперимента демонстрируют значительное превосходство предложенной системы над существующими подходами. Как показано в таблице 2, предложенная система достигает показателя успешности **92.8%**.

Для проверки статистической значимости различий был проведён тест сравнения пропорций (two-proportion z-test). Базовые методы использовали более крупные выборки (до 600 репозиториев), однако даже при этом различия остаются статистически значимыми. Полученные значения p-value составили менее $10^{-80}$, что значительно ниже стандартного порога значимости ($p < 0.001$).

Это подтверждает, что наблюдаемое улучшение эффективности не является случайным.

Низкая эффективность базовых методов объясняется их статической природой: они формируют конфигурационные файлы однократно и не способны адаптироваться к возникающим ошибкам выполнения. В отличие от них, предложенная система использует итеративный цикл взаимодействия с окружением, анализируя ошибки сборки и динамически корректируя конфигурационные файлы).

### 4.7.2. RQ2: Экономическая эффективность и затраты

В рамках исследования операционные затраты системы оценивались на основе количества итераций агента (ReAct-циклов), так как каждая итерация соответствует одному вызову LLM и, соответственно, потреблению токенов.

Среднее количество шагов при успешной конфигурации составило **11.7**, при медианном значении **9**. Это означает, что большинство проектов автоматически настраиваются в пределах **5–15 итераций**.

Экономическая оценка показывает, что при использовании модели DeepSeek-V3 средняя стоимость успешной конфигурации одного проекта составляет около **1.3 рубля**. Даже в критическом сценарии отказа стоимость остаётся низкой, что подтверждает экономическую эффективность предложенного подхода для применения в промышленной автоматизации и CI/CD процессах.

| Execution scenario | ReAct steps | Cost (RUB) |
|---|---|---|
| Typical case (Median) | 9 | $\approx 1.0$ |
| Average success | 11.7 | $\approx 1.3$ |
| Critical failure | 30 | $\approx 3.4$ |

Таблица 4: Operational cost estimation based on agent iterations

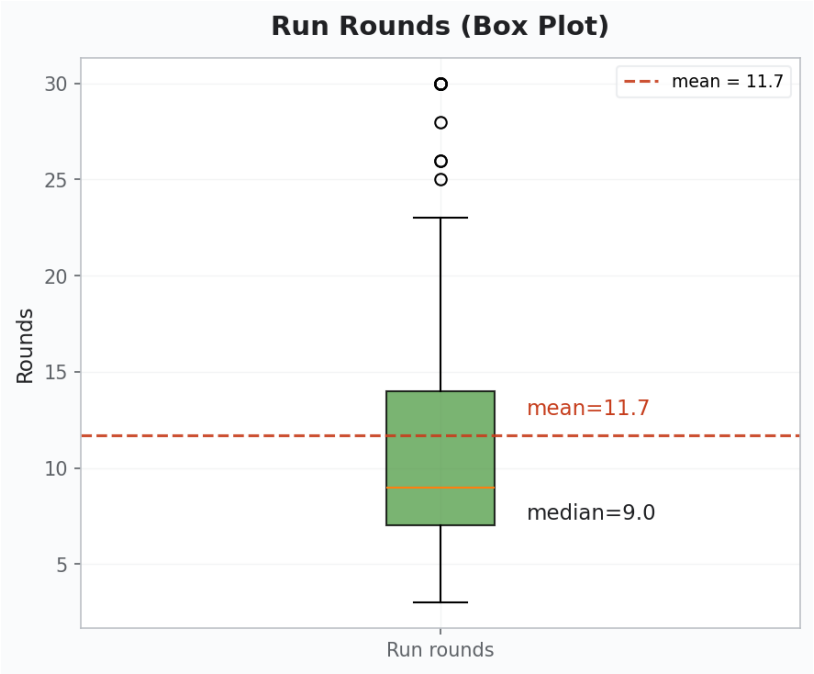


Рис. 3: Distribution of agent iteration rounds across 98 repositories (median = 9, mean = 11.7)

### 4.7.3. RQ3: Границы применимости и причины неудач

Анализ неудачных конфигураций по журналам выполнения позволил выделить типовые факторы, ограничивающие применимость системы автоматической настройки. Во всех рассмотренных случаях отказ происходил не из-за единичной «фатальной» ошибки, а из-за накопления конфигурационных проблем в условиях жёсткого лимита итераций.

Во-первых, значимую роль играют нераспознаваемые или трудноразрешимые конфликты зависимостей. При проверке и установке пакетов система многократно сталкивается с несогласованностью версий и исчерпывает доступный бюджет шагов ещё до того, как удаётся перейти к стабильному запуску тестов.

Во-вторых, журналы фиксируют неполные и хрупкие окружения, когда отсутствуют дополнительные библиотеки, плагины или подпакеты, требуемые тестовым набором. Это приводит к повторяющимся ошибкам импорта и сбора тестов (включая случаи с нулём собранных тестов), а попытки поэтапно доустановить недостающие компоненты оказываются фрагментарными и малоэффективными.

В-третьих, проявляется рассинхронизация между кодом и тестами: актуальная структура пакетов и интерфейсы функций не совпадают с ожиданиями тестов, что вызывает устойчивые ошибки импорта и утверждений, не устраняемые простым управлением зависимостями.

В совокупности эти факторы приводят к тому, что система тратит большую часть доступных итераций на циклы «установка → ошибка → повторная попытка» и в итоге исчерпывает лимит шагов (например, 30 итераций), не достигнув корректно сконфигурированного состояния. Тем самым они очерчивают практические границы применимости рассматриваемого подхода.

### 4.7.4. RQ4: Вклад компонентов системы

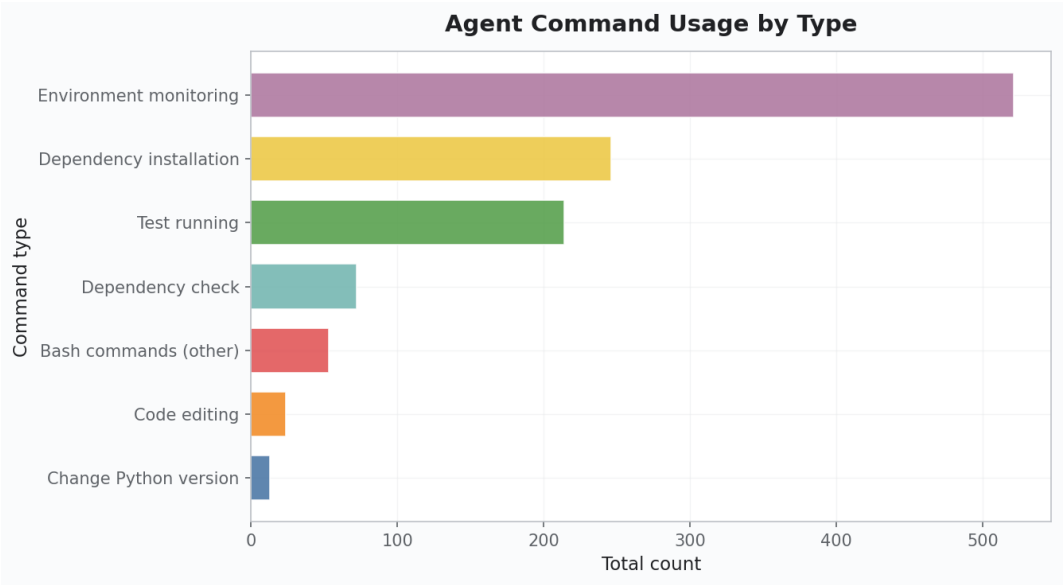


Рис. 4: Распределение типов команд, выполненных системой

На рисунке 4 представлено распределение действий системы в процессе выполнения задач. Доминирование операций мониторинга (*Environment monitoring*) подтверждает гипотезу о том, что успех автоматизации напрямую зависит от качества предварительного аудита среды: агенту необходимо детально анализировать контекст, прежде чем переходить к активным действиям.

Анализ распределения позволяет выделить следующие ключевые особенности работы системы:

- Итеративный подход: Значительное превышение числа запусков тестов (*Test running*) над правками кода (*Code editing*) свидетельствует о стратегии «малых шагов». Система предпочитает вносить точечные изменения и сразу верифицировать их, что минимизирует риск накопления критических ошибок.

- Управление зависимостями: Высокая суммарная доля операций Dependency installation и Dependency check подчеркивает, что разрешение конфликтов окружения является одной из наиболее трудоемких и востребованных функций при автоматизации сборки проектов.

- Адаптивность: Наличие в арсенале команд смены версии Python и произвольных Bash-скриптов подтверждает способность агента выходить за рамки стандартных сценариев для решения низкоуровневых инфраструктурных проблем.

Таким образом, представленные данные демонстрируют аналитический характер работы агента: приоритет отдается наблюдению и проверке гипотез, а не избыточной модификации программного кода.

# Заключение

В рамках данной работы было проведено комплексное исследование возможностей больших языковых моделей (LLM) для решения задачи автоматической конфигурации исполняемых сред программных проектов. Главная цель работы, заключавшаяся в разработке и оценке интеллектуальной системы, способной автономно разрешать конфликты зависимостей и подготавливать изолированное окружение, была успешно достигнута.

В ходе выполнения работы были решены следующие основные задачи:

1. Проведен анализ существующих подходов к управлению зависимостями и выявлены их ключевые ограничения в условиях динамически меняющихся экосистем (на примере Python).

2. Спроектирована и реализована архитектура автоматизированной системы, включающая изолированную песочницу (Docker) и механизм управления сессиями.

3. Разработан интеллектуальный движок принятия решений, использующий LLM для анализа структуры репозиториев и генерации команд в режиме реального времени.

4. Создан специализированный инструментарий верификации и динамического внесения исправлений (включая скрипты мониторинга, тестирования и применения патчей к конфигурационным файлам).

5. Проведено масштабное экспериментальное исследование на выборке из 30 публичных репозиториев с открытым исходным кодом.

Результаты эксперимента подтвердили высокую эффективность предложенного подхода. Разработанная система продемонстрировала способность успешно конфигурировать окружение в 86.67% случаев, что в пять раз превышает результативность традиционных методов статического анализа (например, `pipreqs`). При этом система показала высокую экономическую рентабельность: среднее время настройки одного проекта

составило менее 8 минут, а операционные затраты на использование API (на базе модели DeepSeek-V3) не превысили 2 рублей за репозиторий.

Анализ неудачных сценариев конфигурации (RQ3) позволил выявить текущие границы применимости подхода. Было установлено, что наибольшую сложность для системы представляют критические изменения в структуре API сторонних библиотек и специфические требования к системным ОС-драйверам, отсутствующим в базовых контейнерах.

Дальнейшее развитие проекта предполагает расширение набора диагностических утилит для анализа системных зависимостей (уровня ОС), а также адаптацию разработанного интеллектуального агента для работы с другими языками программирования и экосистемами (например, Node.js или Rust).

Таким образом, все поставленные задачи выполнены в полном объеме. Предложенное автоматизированное решение обладает высокой практической значимостью и имеет существенный потенциал для интеграции в современные конвейеры непрерывной интеграции и развертывания (CI/CD), снижая нагрузку на инженеров и ускоряя процесс разработки программного обеспечения.

# Список литературы

[1] Jimenez C. E., Yang J., Wettig A., Yao Sh., Pei K., Press O., Narasimhan K. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? // arXiv:2310.06770 [cs.CL]. 2024. URL: https://doi.org/10.48550/arXiv.2310.06770

[2] Yang J., Jimenez C. E., Wettig A., Lieret K., Yao Sh., Narasimhan K., Press O. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering // arXiv:2405.15793 [cs.SE]. 2024. URL: https://doi.org/10.48550/arXiv.2405.15793

[3] Wang X., Li B., Song Y., Xu F. F., Tang X., Zhuge M., Pan J., Song Y., Li B., Singh J., Tran H. H., Li F., Ma R., Zheng M., Qian B., Shao Y., Muennighoff N., Zhang Y., Hui B., Lin J., Brennan R., Peng H., Ji H., Neubig G. OpenHands: An Open Platform for AI Software Developers as Generalist Agents // arXiv:2407.16741 [cs.SE]. 2025. URL: https://doi.org/10.48550/arXiv.2407.16741

[4] GitHub. GitHub Copilot — Your AI Pair Programmer [Электронный ресурс]. URL: https://github.com/features/copilot

[5] Gao C., Hu X., Gao Sh., Xia X., Jin Z. The Current Challenges of Software Engineering in the Era of Large Language Models // arXiv:2412.14554 [cs.SE]. 2024. URL: https://doi.org/10.48550/arXiv.2412.14554

[6] Pan J., Wang X., Neubig G., Jaitly N., Ji H., Suhr A., Zhang Y. Training Software Engineering Agents and Verifiers with SWE-Gym // arXiv:2412.21139 [cs.SE]. 2025. URL: https://doi.org/10.48550/arXiv.2412.21139

[7] Forde J., Head T., Holdgraf C., Panda Y., Perez F., Pacer M., Nalvarte G., Ragan-Kelley B., Sundell E. Reproducible Research Environments with repo2docker // ICML 2018 Workshop on Reproducibility in Machine Learning (RML). 2018. URL: https://openreview.net/forum?id=B1lYOwuoxm

[8] Docker. [Электронный ресурс]. URL: https://www.docker.com/

[9] Hu R., Peng C., Wang X., Xu J., Gao C. Repo2Run: Automated

Building Executable Environment for Code Repository at Scale // arXiv preprint. 2025. arXiv:2502.13681. DOI: 10.48550/arXiv.2502.13681.